ML Exam Cheat Sheet

Lab 1 – Validation

distance / predict / MSE

```
def compute_distance(x, y):
    return np.sqrt(x**2 + y**2)

def predict(distance, poly_coeffs):
    return np.polyval(poly_coeffs, distance)

def evaluate(distance, SNR, poly_coeffs):
    return np.mean((predict(distance,poly_coeffs)-SNR)**2)
```

train/test split (random)

```
def separate_test(distance, SNR, test_points):
    idx = np.random.permutation(len(distance))
    ts, tr = idx[:test_points], idx[test_points:]
    return distance[tr], SNR[tr], distance[ts], SNR[ts]
```

K-fold CV (pick best degree)

```
def k_fold_cross_validation(x_train, y_train, k, max_degree):
    idx = np.random.permutation(len(x_train))
    x_fold = np.array_split(x_train[idx], k)
    y_fold = np.array_split(y_train[idx], k)
    results, best, best_perf = [], None, float('inf')
    for deg in range(max_degree+1):
        errs = []
        for i in range(k):
            xt = np.concatenate(x_fold[:i]+x_fold[i+1:])
            yt = np.concatenate(y_fold[:i]+y_fold[i+1:])
            model = fit(xt, yt, deg)
            errs.append(evaluate(x_fold[i], y_fold[i], model))
        mean_err = np.mean(errs)
        results.append(mean_err)
        if mean_err < best_perf:
            best_perf, best = mean_err, fit(x_train, y_train, deg)
    return best, results
```

Tikhonov reg. (add after fit)

```
def evaluate_tikhonov(x_train, y_train, lambda_par, max_degree):
    results, best, best_perf = [], None, float('inf')
    for deg in range(max_degree+1):
        model = np.polyfit(x_train, y_train, deg)
        loss = evaluate(x_train,y_train,model) + lambda_par*np.sum(model**2)
        results.append(loss)
        if loss < best_perf:
            best_perf, best = loss, model
    return best, results
```

MDL reg. (penalty $\propto 2^{deg+1}$)

```
def evaluate_representation(x_train, y_train, lambda_par, max_degree):
    results, best, best_perf = [], None, float('inf')
    for deg in range(max_degree+1):
        model = np.polyfit(x_train, y_train, deg)
        loss = evaluate(x_train,y_train,model) + (2**(deg+1))*lambda_par
        results.append(loss)
        if loss < best_perf:
            best_perf, best = loss, model
    return best, results
```

Lab 2A – Perceptron

homogeneous coords

```
def to_homogeneous(X_training, X_test):
    Xh_training = np.insert(X_training, 0, 1, axis=1)
    Xh_test = np.insert(X_test, 0, 1, axis=1)
    return Xh_training, Xh_test
```

count_errors (used everywhere)

```
def count_errors(current_w, X, Y):
    idx = np.where(np.sign(X @ current_w) != Y)[0]
    n, index = (len(idx), idx) if len(idx) > 0 else (0, -1)
    return n, index
```

Fixed-order perceptron

```
def perceptron_fixed_update(current_w, X, Y):
    n, idx = count_errors(current_w, X, Y)
    if n > 0:
        return current_w + Y[idx[0]] * X[idx[0]]
    return current_w

def perceptron_no_randomization(X, Y, max_num_iterations):
    curr_w = np.zeros(X.shape[1]); best_w = curr_w.copy()
    n_err, _ = count_errors(curr_w, X, Y); best_err = n_err/len(X)
    for _ in range(max_num_iterations):
        if n_err == 0: break
        curr_w = perceptron_fixed_update(curr_w, X, Y)
        n_err, _ = count_errors(curr_w, X, Y)
        err = n_err/len(X)
        if err < best_err:
            best_err, best_w = err, curr_w.copy()
    return best_w, best_err
```

Randomized perceptron

```
def perceptron_randomized_update(current_w, X, Y):
    n, idx = count_errors(current_w, X, Y)
    if n > 0:
        r = np.random.choice(idx)
        return current_w + Y[r] * X[r]
    return current_w

def perceptron_with_randomization(X, Y, max_num_iterations):
    curr_w = np.zeros(X.shape[1]); best_w = curr_w.copy()
    n_err, _ = count_errors(curr_w, X, Y); best_err = n_err/len(X)
    for _ in range(max_num_iterations):
        if n_err == 0: break
        curr_w = perceptron_randomized_update(curr_w, X, Y)
        n_err, _ = count_errors(curr_w, X, Y)
        err = n_err/len(X)
        if err < best_err:
            best_err, best_w = err, curr_w.copy()
    return best_w, best_err
```

Test loss

```
def loss_estimate(w, X, Y):
    n_err, _ = count_errors(w, X, Y)
    return n_err/len(Y)
```

Lab 2B – Regression

Least squares (normal eq.)

```
def least_squares(X_matrix, labels):
    # solves X.T@X @ coeff = X.T@labels
    return np.linalg.solve(X_matrix.T @ X_matrix, X_matrix.T @ labels)

def evaluate_model(x, y, coeff):
    return np.mean((x @ coeff - y)**2)
```

Ridge / regularized LS

```
def regularized_least_squares(X_matrix, labels, lambda_par):
    A = X_matrix.T @ X_matrix
    b = X_matrix.T @ labels
    return np.linalg.solve(A + lambda_par*np.eye(len(A)), b)
```

K-fold CV over λ

```
def K_fold(X_training, Y_training, lambda_vec, K):
    idx = np.random.permutation(len(X_training))
    x_folds = np.array_split(X_training[idx], K)
    y_folds = np.array_split(Y_training[idx], K)
    best_model, best_perf = None, float('inf')
    results, models = [], []
    for lam in lambda_vec:
        errs = []
        for i in range(K):
            xt = np.concatenate(x_folds[:i]+x_folds[i+1:])
            yt = np.concatenate(y_folds[:i]+y_folds[i+1:])
            model = regularized_least_squares(xt, yt, lam)
            errs.append(evaluate_model(x_folds[i], y_folds[i], model))
        mean_errs = np.mean(errs); results.append(mean_errs)
        full_model = regularized_least_squares(X_training, Y_training, lam)
        models.append(full_model)
        if mean_errs < best_perf:
            best_perf, best_model = mean_errs, full_model
    return best_model, best_perf, models, results
```

Lab 3 – Soft SVM (SGD)

margin / outliers

```
def count_outliers(current_w, X, Y):
    margin = (X @ current_w) * Y
    return np.sum(margin < 1), np.sum(margin < 0) # (outliers, misclassified)

def find_margin(current_w):
    return 1/max(1e-9, np.linalg.norm(current_w))
```

SGD soft-SVM (avg. last iters)

```
def sgd_soft_svm(X, Y, lambda_par, max_num_iterations, averaging_iterations):
    theta = np.zeros(X.shape[1]); best_w = np.zeros(X.shape[1])
    for t in range(max_num_iterations):
        current_w = theta / (lambda_par*(t+1))
        i = np.random.randint(len(X))
        if (X[i] @ current_w) * Y[i] < 1:
            theta += X[i]*Y[i]
        if t >= max_num_iterations - averaging_iterations:
            best_w += current_w/averaging_iterations
    margin = find_margin(best_w)
    outliers, misclassified = count_outliers(best_w, X, Y)
    return best_w, margin, outliers, misclassified
```

K-fold CV over λ (SVM)

```
def K_fold(X_training, Y_training, lambda_vec, K):
    idx = np.random.permutation(len(X_training))
    x_fold = np.array_split(X_training[idx], K)
    y_fold = np.array_split(Y_training[idx], K)
    best, best_perf, models, results = None, float('inf'), [], []
    for lam in lambda_vec:
        errs = []
        for i in range(K):
            xt = np.concatenate(x_fold[:i]+x_fold[i+1:])
            yt = np.concatenate(y_fold[:i]+y_fold[i+1:])
            model, *_ = sgd_soft_svm(xt, yt, lam, 100000, 10000)
            _, mis = count_outliers(model, x_fold[i], y_fold[i])
            errs.append(mis/len(x_fold[i]))
        mean_errs = np.mean(errs); results.append(mean_errs)
        full_model, *_ = sgd_soft_svm(X_training, Y_training, lam, 100000, 10000)
        models.append(full_model)
        if mean_errs < best_perf:
            best_perf, best = mean_errs, full_model.copy()
    return best, best_perf, models, results
```

Lab 5 – Random Forest

Tree.classify (recursive)

```
def classify(self, x):
    if self.leaf != 0:
        return self.leaf
    return (self.right.classify(x) if x[self.idx] > self.thresh
            else self.left.classify(x))
```

ID3: best split + recurse

```
# best_idx=-1, best_thresh=-1, best_entropy=1e9 given
for idx in range(X.shape[1]):
    values = np.unique(X[:, idx])
    for j in range(len(values)-1):
        thresh = (values[j]+values[j+1])/2
        left_Y = Y[X[:, idx] <= thresh]
        right_Y = Y[X[:, idx] > thresh]
        H = Tree.entropy(left_Y, right_Y)
        if H < best_entropy:
            best_entropy, best_idx, best_thresh = H, idx, thresh
# ... (leaf/no-split check already given) ...
self.idx, self.thresh = best_idx, best_thresh
self.left, self.right = Tree(), Tree()
self.left.id3_training(X[left_samples], Y[left_samples], max_depth-1, printing)
self.right.id3_training(X[right_samples], Y[right_samples], max_depth-1, printing)
```

Extra-Trees: random threshold split

```
for idx in range(X.shape[1]):
    values = np.unique(X[:, idx])
    if len(values) > 1:
        thresh = np.random.uniform(np.min(values), np.max(values))
        left_Y = Y[X[:, idx] <= thresh]
        right_Y = Y[X[:, idx] > thresh]
        H = Tree.entropy(left_Y, right_Y)
        if H < best_entropy:
            best_entropy, best_idx, best_thresh = H, idx, thresh
# then same recursion as ID3 but calling extra_training
```

Forest: vote + bagging

```
def classify(self, x): # majority vote
    return np.sign(sum(tree.classify(x) for tree in self.forest))

def bag(self, X, Y): # bootstrap + random feature subset
    idx = np.random.choice(len(X), self.samples, replace=True)
    X_bagged, Y_bagged = X[idx], Y[idx]
    keep = np.random.choice(X.shape[1], self.features, replace=False)
    for i in range(X.shape[1]):
        if i not in keep:
            X_bagged[:, i] = 0
    return X_bagged, Y_bagged
```

Lab 6A – Clustering

K-means

```
def kmeans(points, k, max_iters=50):
    idx = np.random.choice(len(points), k, replace=False)
    centroids = points[idx].copy()
    clusters = np.zeros(len(points), dtype=int)
    error = []
    for _ in range(max_iters):
        dist = np.linalg.norm(points[:, None]-centroids, axis=2)
        new_clusters = np.argmin(dist, axis=1)
        if np.array_equal(new_clusters, clusters):
            break
        clusters = new_clusters
        for i in range(k):
            pts = points[clusters == i]
            if len(pts) > 0:
                centroids[i] = np.mean(pts, axis=0)
        cost = np.sum((points-centroids[clusters])**2)/len(points)
        error.append(cost)
    return centroids, clusters, error
```

DBSCAN

```
def dbscan(points, epsilon, M):
    dists = distance_matrix(points)
    cl = -np.ones(len(points)) # -1 unassigned, 0 outlier, >=1 cluster
    C = 0
    for i in range(len(points)):
        if cl[i] != -1: continue
        nbs = list(find_neighbors(dists[i], epsilon))
        if len(nbs) < M:
            cl[i] = 0 # outlier
        else:
            C += 1; cl[i] = C
            for n in nbs:
                if cl[n] <= 0:
                    unassigned = (cl[n] == -1)
                    cl[n] = C
                    nn = find_neighbors(dists[n], epsilon)
                    if unassigned and len(nn) >= M:
                        nbs.extend(nn)
    return cl
```

Lab 6B – Neural Networks

sklearn MLPClassifier

```
mlp = MLPClassifier(hidden_layer_sizes=size, max_iter=200,
                    learning_rate_init=0.1)
mlp.fit(X_training, Y_training)
train_acc = mlp.score(X_training, Y_training)
test_acc = mlp.score(X_test, Y_test)
# mlp.loss_curve_ -> list of loss per iteration
```

Activation (DIY MLP)

```
def __call__(self, x): # forward
    if self.name == 'relu':
        return np.maximum(0, x), x
    elif self.name == 'sigmoid':
        return 1/(1+np.exp(-x)), x

def derivative(self, x):
    if self.name == 'relu':
        return np.where(x > 0, 1, 0)
    elif self.name == 'sigmoid':
        s = 1/(1+np.exp(-x))
        return s*(1-s)
```

DiyMlp.forward

```
for l in range(1, self.n_layers):
    prev = cp(x) if l == 1 else out
    z = self.params[f'W{l}'] @ prev + self.params[f'b{l}']
    linear_cache = (prev, self.params[f'W{l}'], self.params[f'b{l}'])
    out, activation_cache = self.params[f'activation{l}'](z)
    caches.append((linear_cache, activation_cache))
return out, caches
```

Train / predict DIY MLP

```
diy_mlp = DiyMlp([784, 40, 10], learning_rate)
diy_mlp.train(X_training, Y_training, epochs=200)
Y_pred = diy_mlp.predict(X_test)
acc = np.mean(Y_pred == Y_test)
```

Lab 7 – Reinforcement Learning

Policy evaluation (Bellman)

```
for s in range(env.nS):
    v = 0
    for a, action_prob in enumerate(policy[s]):
        v += action_prob * np.sum(
            env.P[s, a] * (env.R[s, a] + discount_factor*V))
    delta = max(delta, abs(v - V[s]))
    V[s] = v
```

Epsilon-greedy action (Q-learn & SARSA)

```
def choose_action(self, state):
    if np.random.rand() < self.epsilon:
        return np.random.choice(self.env.nA)
    return np.argmax(self.Q[state])
```

Q-Learning update (off-policy)

```
def learn(self, state, action, reward, next_state):
    td_target = reward + self.gamma*np.max(self.Q[next_state])
    self.Q[state, action] += self.lr*(td_target - self.Q[state, action])
```

SARSA update (on-policy)

```
def learn(self, state, action, reward, next_state, next_action):
    td_target = reward + self.gamma*self.Q[next_state, next_action]
    self.Q[state, action] += self.lr*(td_target - self.Q[state, action])
```